# Integration and Progress

## Representative end-to-end workflow

Borrower login -> catalog and interval search -> request form -> Django validation -> transactional service -> SQLite Pending reservation -> authorized reviewer approval -> persisted Approved status -> borrower status page and updated availability. This is a functioning initial reservation workflow; it does not include physical checkout or return.

## Components and integration

Django authentication/session middleware identifies users. Forms convert local browser dates to timezone-aware values and reject invalid inputs. Views enforce route-level authorization, services enforce business rules, ORM models persist relationships, and templates display status and errors. SQLite IMMEDIATE transactions serialize conflicting approvals.

The existing Equipment admin and new Reservation model share the same database. Deactivating equipment retains history and blocks new requests/approval. Reviewer identity and review time are recorded. Reservation foreign keys use PROTECT to preserve linked records.

## Start the application

From the package root, create/activate a Python 3.12 virtual environment and install src/campus_checkout/requirements.txt. Then change to src/campus_checkout and run: python manage.py migrate; python manage.py createsuperuser; python manage.py create_demo_borrower; python manage.py runserver. Execute each command separately.

Choose your own passwords. The demo-borrower command creates a fictional borrower and demo laptop. It refuses to overwrite an existing account. Open http://127.0.0.1:8000/. The administrator account can manage inventory at /admin/ and review requests at /review/. Full commands are in START_WEEKS_2_3.md.

# Integration and Progress / continued

## Manual demonstration to perform

1. Log in as demo-borrower. Search Laptop with a start tomorrow and an end one hour later. Click Request and enter that interval; submit. My requests displays Pending.

2. Log out and log in as your administrator. Open Review requests and approve the request. Log out, return as demo-borrower, and confirm Approved in My requests. Search the same interval and confirm Unavailable.

3. Submit an end before the start; confirm the visible validation message and no new request. Create another valid request overlapping the approved booking; try approving it as administrator. Confirm the conflict error and unchanged Pending state.

4. A non-superuser reviewer needs the explicit inventory.review_reservation permission assigned through admin Users. A borrower's direct approval POST must return 403. Adjacent approved intervals are allowed.

## Executed evidence and persistence

The assistant executed manage.py demo_workflow on a separate SQLite database on October 27. It invokes the real Django routes with the Django test client and synthetic accounts. This is HTTP/template integration evidence, not a claimed manual Chrome or Safari session.

Observed: invalid input returned HTTP 200 with a form error and zero created reservations; valid input returned 302 and persisted Pending; borrower approval returned 403; reviewer approval returned 302 and persisted Approved with reviewer identity. Borrower HTML contained Approved, and availability HTML contained Unavailable.

The recorded reservation was #1, equipment EVID-8B4BDAA5, from 2026-10-29 18:26:40 UTC to 19:26:40 UTC. A fresh management-shell process reopened the same database and confirmed Approved plus the borrower/equipment links. This verifies reservation persistence across process restart, not checkout persistence.

Raw evidence: verification/workflow_output.txt, persistence.txt, migrations.txt and provenance.txt. The PermissionDenied traceback in workflow output is the expected logged 403, not a failed demonstration.

# Integration and Progress / continued

## Tests and requirement status

python manage.py test inventory --noinput --verbosity 2: 21 tests passed. See verification/test_output.txt. The integrated test verifies invalid input, successful request, approval, owner-visible status, and unavailable catalog state; the concurrent test repeats competing approvals ten times. Django system check found no issues; migration drift check found no changes.

| Requirement | Status | Scope / remaining work |
|---|---|---|
| FR-01 | Implemented | Login, logout, credential rejection |
| FR-02 | Partial | Current routes secured; lending pending |
| FR-03 | Implemented | Inventory and inactive item guard |
| FR-04 | Implemented | Search and interval availability |
| FR-05 | Implemented | Validated Pending requests |
| FR-06 | Implemented | Approval/rejection with conflict checks |
| FR-07 | Partial | Own status only; cancellation planned |
| FR-08 to 10 | Planned | Checkout, return, overdue |
| FR-11 | Partial | Reservation persistence; loans pending |
| FR-12 | Planned | Usage counts by date range |

# Integration and Progress / continued

## Design changes and unresolved issues

New templates and transactional services extend the original monolithic Django design. File-backed SQLite testing and IMMEDIATE write transactions address concurrent approvals. No production deployment claim is made. Validation errors are displayed inline or through Django messages.

Unresolved: no lending lifecycle, cancellation, measured NFR-03 workload performance, real-browser compatibility evidence, or complete keyboard/accessibility review. Availability describes approved time reservations, not physical possession of equipment after a late return.

The remaining prototype uses DEBUG and a public local development secret fallback. Deployment requires separate security/configuration work. Synthetic demonstration accounts contain no real student data.

## Next five development tasks

1. Add owner-only cancellation with state rules and negative authorization tests. 2. Add checkout records and prevent duplicate/open-item checkout, including late returns. 3. Add idempotent return and overdue queries. 4. Add date-range usage counts verified against a hand-counted fixture. 5. Run workload/browser/accessibility evaluation and prepare the final demonstration and report.